Projeto final de automação IoT

Etapa 4: integração IoT, dados e controle remoto

Monitor IoT de temperatura para servidor
Engenharia de Software · Internet das Coisas
Católica de Santa Catarina, Centro Universitário
Professor: Msc. Rodrigo Hagstrom

Este documento mostra como o protótipo conversa com o servidor: conexão à rede, telemetria, formato das mensagens, banco de dados, painel, comandos remotos e tratamento de falhas. O caminho completo, do ESP32 até o painel e do painel de volta ao ESP32, foi testado no simulador Wokwi em 09/11/2026 (seção 8 e Anexos A e B). Na aula de 13/11, o teste foi repetido com o ESP32 na bancada (seção 8 e Anexos C a F). As mudanças em relação à Etapa 2 estão na seção 9.

Integrantes (Grupo 2): Bruno Ruan dos Santos, Bernardo Wehmuth, João Vítor Dal-Ri, João Vítor Círico, Nicolas Soares Oliveira e Vinícius Bueno de Oliveira.

Data de entrega: 13/11/2026

1. Visão geral da integração

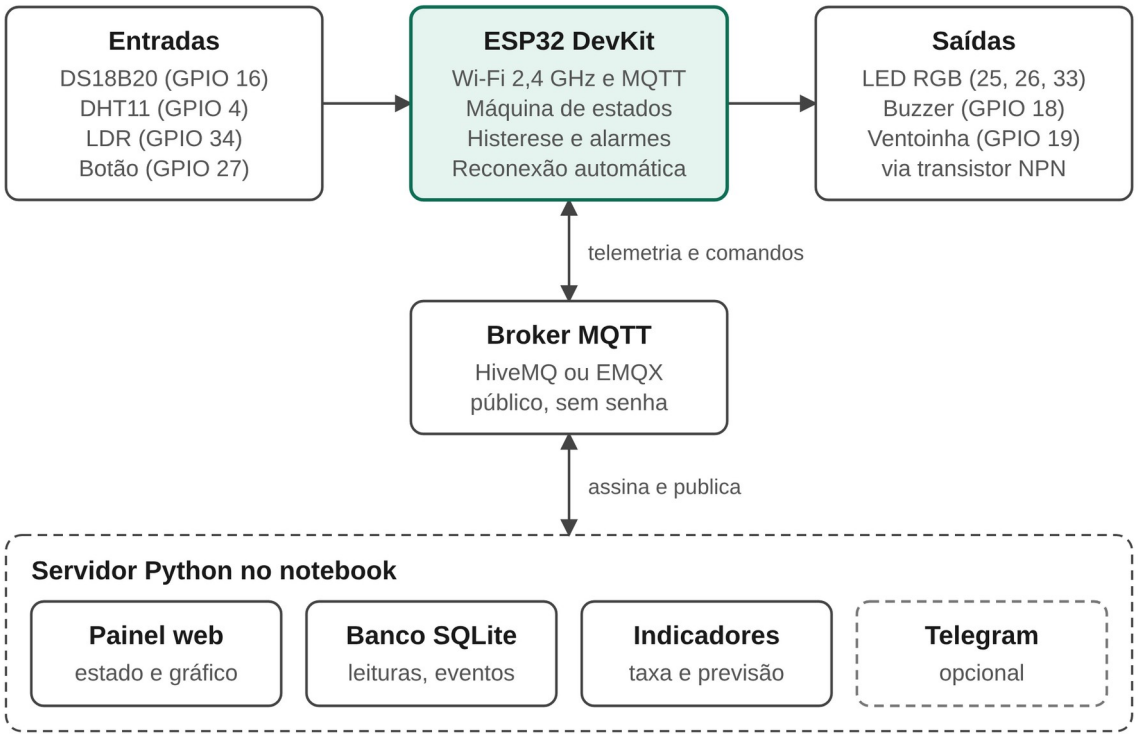


Figura 1. Diagrama de blocos da solução, o mesmo da Etapa 2.

O ESP32 publica a telemetria no broker público HiveMQ (`broker.hivemq.com`, porta 1883). O servidor em Python roda no notebook: assina os tópicos, valida cada mensagem, grava no SQLite e entrega a API e o painel web. Os comandos do painel fazem o caminho de volta pelo mesmo broker. A decisão de ligar a ventoinha continua no ESP32, e o controle local não depende da rede.

Na bancada, o sensor principal foi o DHT11, porque as peças compradas não chegaram a tempo.

2. Conexão à rede

O ESP32 entra no Wi-Fi de 2,4 GHz em modo estação. O nome e a senha da rede ficam no `config.h`, que não vai para o git. No Wokwi, a rede é a `Wokwi-GUEST`, sem senha. Na bancada, o ESP32 usou a rede da sala. Num teste de 06/11 com o sketch `wifi_check`, o ESP32 conectou à rede da sala em 3 s e os dois brokers públicos responderam na porta 1883.

Com o Wi-Fi conectado, o ESP32 abre a conexão MQTT com a biblioteca PubSubClient. O identificador do cliente no broker é `esp32-rack01` com um sufixo aleatório de quatro dígitos hexadecimais. Ao conectar, o ESP32 publica `online` no tópico `status`, com a mensagem retida, e assina os tópicos de comando (`cmd/#`). Na mesma conexão ele deixa registrado o Last Will: se a conexão cair sem aviso, o próprio broker publica `offline` nesse tópico.

3. Telemetria e formato das mensagens

Todos os tópicos começam por `thermomon/g2-axr2adzj/rack01/`. O código `g2-axr2adzj` junta o número do grupo a um sufixo aleatório, para não colidir com tópicos de outras pessoas no broker público. A tabela mostra o final de cada tópico.

Tópico	Direção	Conteúdo	Observação
`telemetry`	ESP32 → broker	JSON com os campos abaixo	A cada 5 s, com QoS 0. Começa depois que o primeiro estado é calculado.
`status`	ESP32 → broker	`online` ou `offline`	Mensagem retida. O `offline` é publicado pelo broker (Last Will) quando a conexão cai.
`cmd/mode`	Servidor → ESP32	`auto` ou `manual`	Alterna o modo de operação.
`cmd/fan`	Servidor → ESP32	`on` ou `off`	Só vale no modo manual e nunca desliga a ventoinha em crítico ou falha.
`cmd/limits`	Servidor → ESP32	`{"warning":27,"critical":32}`	Valores de 20 a 50 °C, com o crítico pelo menos 3 °C acima do atenção.
`cmd/silence`	Servidor → ESP32	`1`	Silencia o buzzer do estado atual.

Exemplo de mensagem de telemetria:

```
{"device":"esp32-rack01","uptime_s":1234,"temp_c":24.31,"dht_temp_c":24.0,
"humidity_pct":41,"lid_open":false,"fan_on":true,"state":"warning",
"mode":"auto","warning_c":27.0,"critical_c":32.0}
```

Campo	Tipo	Descrição
`device`	texto	Identificação do dispositivo: `esp32-rack01`.
`uptime_s`	inteiro	Segundos desde a partida do ESP32.
`temp_c`	número	Temperatura do sensor principal (DS18B20 ou DHT11, conforme o `config.h`), em °C. Vai `null` se a leitura falhar.
`dht_temp_c`	número	Temperatura do DHT11, em °C. Vai `null` se a leitura falhar.
`humidity_pct`	inteiro	Umidade relativa do DHT11, em %. Vai `null` se a leitura falhar.
`lid_open`	booleano	Verdadeiro quando a leitura do LDR fica abaixo do limiar (tampa aberta).
`fan_on`	booleano	Verdadeiro quando a ventoinha está ligada.

Campo	Tipo	Descrição
`state`	texto	`normal`, `warning`, `critical` ou `fault`.
`mode`	texto	`auto` ou `manual`.
`warning_c`	número	Limite de atenção em vigor no ESP32, em °C.
`critical_c`	número	Limite crítico em vigor no ESP32, em °C.

O campo `device` identifica o ESP32. Ele não acerta o relógio pela rede e envia só o `uptime_s`. A data e a hora vêm do servidor, que grava o instante de chegada de cada mensagem em UTC, no formato ISO 8601. O painel mostra esse horário convertido para o horário local.

4. Servidor e banco de dados

O servidor fica na pasta `server/` do repositório. Ele usa Python 3.10, FastAPI, Uvicorn e paho-mqtt 2.2, com as versões fixadas no `requirements.txt`, e roda num ambiente virtual. O banco é um arquivo SQLite, sem servidor de banco.

Arquivo	O que faz
`main.py`	Cria a aplicação FastAPI, liga o cliente MQTT e entrega o painel.
`config.py`	Lê a configuração de variáveis de ambiente, todas com valor padrão.
`mqtt_client.py`	Assina `telemetry` e `status`, grava leituras e eventos e publica os comandos.
`validation.py`	Valida cada mensagem antes de gravar.
`database.py`	Cria as tabelas e faz as consultas e gravações, sempre com SQL parametrizado.
`indicators.py`	Calcula a tendência e a previsão do tempo até o limite crítico.
`routes.py`	Endpoints da API.

O broker é público, então toda mensagem é tratada como entrada não confiável. A telemetria só é gravada se tiver no máximo 512 bytes, for um JSON válido, tiver exatamente as chaves da seção 3, com os tipos e as faixas certos, e vier do `device` configurado. O `status` só aceita `online` ou `offline`. O que não passa é descartado com uma linha no log, sem derrubar o servidor.

Tabela `readings`, com uma linha por mensagem de telemetria:

Coluna	Tipo	Descrição
`id`	INTEGER	Chave primária, gerada pelo banco.
`device`	TEXT	Identificação do ESP32.
`received_at`	TEXT	Data e hora de chegada ao servidor, em UTC (ISO 8601).
`uptime_s`	INTEGER	Segundos desde a partida do ESP32.
`temp_c`	REAL	Temperatura do sensor principal, em °C. Fica vazia se a leitura falhar.
`dht_temp_c`	REAL	Temperatura do DHT11, em °C. Fica vazia se a leitura falhar.
`humidity_pct`	INTEGER	Umidade relativa do DHT11, em %. Fica vazia se a leitura falhar.
`lid_open`	INTEGER	0 ou 1.
`fan_on`	INTEGER	0 ou 1.
`state`	TEXT	`normal`, `warning`, `critical` ou `fault`.
`mode`	TEXT	`auto` ou `manual`.

Coluna	Tipo	Descrição
`warning_c`	REAL	Limite de atenção em vigor no ESP32.
`critical_c`	REAL	Limite crítico em vigor no ESP32.

Tabela `events`:

Coluna	Tipo	Descrição
`id`	INTEGER	Chave primária, gerada pelo banco.
`device`	TEXT	Identificação do ESP32.
`occurred_at`	TEXT	Data e hora do evento, em UTC (ISO 8601).
`kind`	TEXT	`state_change`, `sensor_fault`, `command`, `online` ou `offline`.
`detail`	TEXT	Texto livre ou JSON com o valor anterior e o novo.

Um evento é gravado quando o estado muda entre duas leituras seguidas, quando o sensor entra ou sai de falha, quando um comando é enviado e quando o `status` muda. O cliente MQTT e a API rodam em threads diferentes, então cada operação abre a própria conexão com o banco, que fica no modo WAL.

Para começar com o banco vazio, o `reset_db.py` pede confirmação, move o banco atual para `server/data/backup/`, com a data e a hora no nome, e cria um banco novo. O painel e a API não têm opção de apagar dados.

O servidor tem 80 testes automatizados com pytest, que cobrem a validação, o banco, o tratamento das mensagens MQTT, as rotas da API, os indicadores e o reset do banco. Todos passam.

Método	Caminho	O que faz
GET	`/api/latest`	Última leitura e se o dispositivo está online.
GET	`/api/readings?minutes=60`	Leituras do período, para o gráfico.
GET	`/api/events?limit=50`	Últimos eventos.
GET	`/api/indicators`	Tendência e previsão.
POST	`/api/commands/mode`	Publica em `cmd/mode`. Corpo `{"mode":"auto"}`.
POST	`/api/commands/fan`	Publica em `cmd/fan`. Corpo `{"fan":"on"}`.
POST	`/api/commands/limits`	Publica em `cmd/limits`. Corpo `{"warning":27,"critical":32}`.
POST	`/api/commands/silence`	Publica em `cmd/silence`. Sem corpo.

5. Painel web

O painel fica na pasta `web/` e é feito em HTML, CSS e JavaScript, sem framework. O próprio servidor entrega a página em http://127.0.0.1:8000, e ela consulta a API a cada 2 segundos. O Anexo B mostra o painel durante o teste no Wokwi. Na tela aparecem:

- A temperatura atual, com a cor e o nome do estado.
- Uma régua com as faixas normal, atenção e crítico e os limites em vigor no ESP32.
- O dispositivo, o status online ou offline e o tempo desde a última leitura.
- A ventoinha, a tampa, o modo, a umidade e a temperatura do DHT.

- O gráfico das leituras dos últimos 15 minutos, 1 hora ou 6 horas, com as linhas dos dois limites.
- A tendência em °C por minuto e a previsão do tempo até o limite crítico.
- A tabela dos últimos eventos e os controles de modo, ventoinha, limites e alarme.

O dispositivo aparece offline quando a última leitura tem mais de 15 s ou quando o último `status` recebido é `offline`. O Chart.js e a fonte ficam no repositório, sem depender de CDN. Os dados recebidos entram na página com `textContent`, nunca como HTML, e o servidor escuta só no próprio notebook (127.0.0.1).

6. Comandos remotos

Os comandos saem do painel, passam pela API e são publicados pelo servidor no broker. Como qualquer pessoa pode publicar nos tópicos de um broker público, a validação acontece dos dois lados:

Comando	Tópico	No painel e no servidor	No ESP32
Modo	`cmd/mode`	Só `auto` ou `manual`.	Aceita só `auto` ou `manual`.
Ventoinha	`cmd/fan`	Só `on` ou `off`. Os botões ficam ativos só no modo manual.	Aceita só `on` ou `off`. Vale no modo manual, e em crítico e falha a ventoinha continua ligada.
Limites	`cmd/limits`	Inteiros de 20 a 50 °C, com o crítico pelo menos 3 °C acima do atenção. Fora disso, o painel não envia e a API responde 400.	Mesma faixa. JSON inválido ou valor fora da faixa faz o comando inteiro ser ignorado, com uma linha no monitor serial.
Silenciar	`cmd/silence`	Sem corpo.	Aceita só `1`.

Sem conexão com o broker, a API responde 503 e não grava nada. Cada comando enviado vira um evento `command` no banco. O painel não muda o valor na hora: a mudança aparece quando a próxima telemetria do ESP32 confirma.

7. Falhas e reconexão

- Wi-Fi fora do ar: o ESP32 escreve `Wi-Fi lost` no monitor serial e tenta conectar de novo a cada 10 s, além da reconexão automática do próprio ESP32.
- Broker fora do ar: nova tentativa de conexão MQTT a cada 5 s. Cada tentativa segura o loop por no máximo 2 s.
- Durante a queda, os sensores, os estados, o LED, o buzzer e a ventoinha continuam funcionando, porque a decisão é local. A telemetria desse intervalo não fica guardada no ESP32 e volta a ser enviada quando a conexão volta.
- Queda do ESP32 sem aviso: o broker publica `offline` no tópico `status` (Last Will). O servidor grava o evento e o painel mostra o dispositivo offline.
- Queda da rede do notebook: o cliente MQTT do servidor reconecta sozinho, com espera de 1 a 30 s entre as tentativas, e a API continua respondendo com o que já está no banco.

No Wokwi, ao parar a simulação, o broker publicou `offline` cerca de 15 s depois. Na bancada, o USB do ESP32 foi desligado: o broker publicou offline 22 s depois da última leitura, e o painel avisou que não chegavam dados novos. Com o USB de volta, o ESP32 ficou online de novo 33 s depois do offline. Ao religar, ele estava com os limites padrão de 27 e 32 °C, porque os limites recebidos por comando não ficam gravados no ESP32. A queda só do Wi-Fi não foi testada e fica para a Etapa 5.

8. Testes de ponta a ponta

Os testes foram feitos em 09/11/2026 com o circuito final no Wokwi, o broker HiveMQ e o servidor e o painel rodando no notebook. A temperatura foi mudada pelo controle do DS18B20 no simulador. Os comandos foram enviados pelo painel e também publicados do notebook direto no broker, para testar valores que o painel não deixa enviar. A última coluna traz o resultado de 13/11, com o ESP32 na bancada e o DHT11 como sensor principal. Lá, a temperatura mudou com o sensor aquecido na mão ou com os limites enviados pelo painel.

#	Situação	Esperado	Obtido no Wokwi (simulação)	Obtido na bancada
1	ESP32 liga com rede	Conecta e publica a telemetria a cada 5 s	Conectou à `Wokwi-GUEST` e ao HiveMQ. O notebook recebeu a telemetria a cada 5 s	Conectou à rede da sala e ao HiveMQ. O servidor registrou o online às 19:35 e recebeu a telemetria a cada 5 s
2	DS18B20 em 22, 24,5, 26, 28,2, 30 e 33,4 °C	O painel mostra o mesmo estado do ESP32	O painel acompanhou o estado em poucos segundos, de normal até crítico	O painel acompanhou normal, atenção e crítico, com o DHT11 aquecido na mão e com os limites mudados pelo painel
3	Silenciar pelo painel em crítico	Buzzer para	Buzzer parou	Buzzer parou
4	Modo manual e ventoinha desligada em crítico	Ventoinha continua ligada	Continuou ligada	Continuou ligada
5	Ventoinha desligada em atenção (29 °C), modo manual	Ventoinha desliga	Desligou	Desligou, e ligou de novo com o comando on
6	Limites 30 e 31 °C no painel	Painel recusa (diferença menor que 3 °C)	Recusado pela validação do painel	Não repetido na bancada
7	Limites 30 e 35 °C pelo painel	ESP32 aceita	Aceito pelo ESP32	Aceitos. Limites como 24 e 28 °C e 28 e 32 °C voltaram na telemetria seguinte
8	Limites 21 e 30 °C publicados do notebook, com 22 °C no sensor	ESP32 aceita e o estado vai para atenção	Aceito. O estado passou para atenção	Feito pelo painel: com 26,7 °C no sensor, os limites 20 e 26 °C levaram o estado a crítico
9	Limites 5 e 90 °C, JSON malformado e modo `turbo`, publicados do notebook	ESP32 ignora	Os três foram ignorados	Não repetido na bancada
10	Volta ao modo automático pelo painel	ESP32 volta ao automático	Voltou ao automático	Voltou ao automático

#	Situação	Esperado	Obtido no Wokwi (simulação)	Obtido na bancada
11	Simulação parada	O broker publica `offline` (Last Will)	`offline` publicado cerca de 15 s depois	USB do ESP32 desligado: offline publicado 22 s depois da última leitura. O ESP32 voltou online 33 s depois do offline, com os limites padrão de 27 e 32 °C
12	Banco depois do teste	Leituras e eventos gravados	62 leituras e os eventos de estado e de comando correspondentes	589 leituras e 70 eventos gravados durante a aula, entre eles a falha de sensor (sensor_fault) e o offline e o online do ESP32
13	Roteador do celular desligado por 20 s	Controle local segue e a telemetria volta	Não testado no Wokwi	Não testado. Fica para a Etapa 5
14	Notebook fora da rede por 20 s	Painel mostra offline e as leituras voltam	Não testado no Wokwi	Não testado. Fica para a Etapa 5

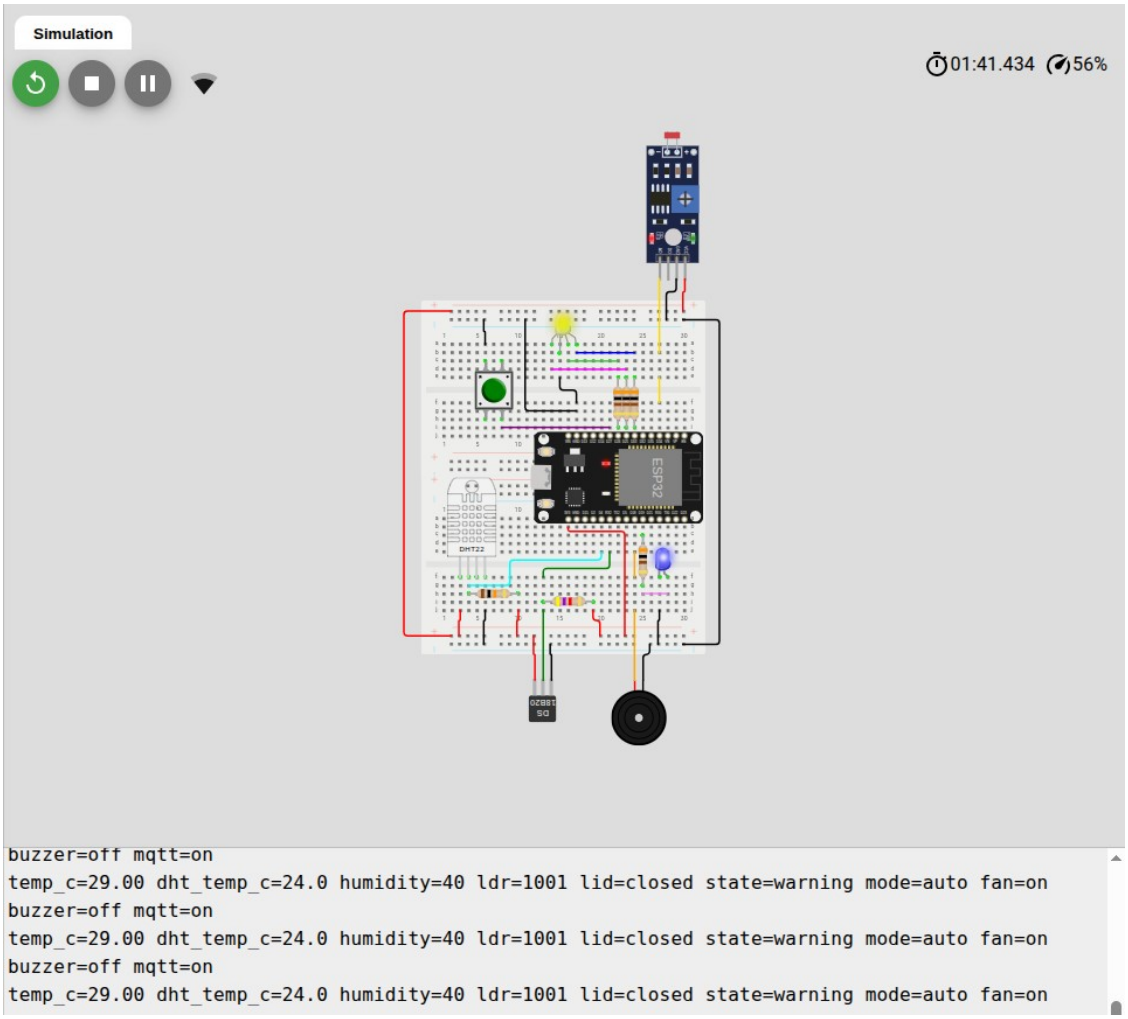
9. Decisões e alterações em relação à Etapa 2

Item	Etapa 2	Etapa 4	Motivo
Telemetria	9 campos	Mais `warning_c` e `critical_c`, os limites em vigor no ESP32	Depois de um `cmd/limits`, o painel e a previsão usam os limites reais, e não os padrões.
Reconexão	Nova tentativa a cada 5 s	MQTT a cada 5 s, com no máximo 2 s por tentativa. Wi-Fi a cada 10 s, além da reconexão automática do ESP32	Sem broker, a tentativa de conexão travava o loop até o tempo limite. Com 2 s, o controle local segue.
Status online e offline	Reforço	Feito, com Last Will e mensagem retida	O painel mostra quando o ESP32 sai do ar.
Offline no painel	Não definido	15 s sem leitura ou `status` igual a `offline`	Com telemetria a cada 5 s, são três mensagens perdidas.
Limpeza do banco	Não prevista	`reset_db.py`, com cópia do banco, sem opção no painel	Para ninguém apagar os dados com um clique durante uma demonstração.
Tendência e previsão	Etapa 5	Já calculadas pelo servidor e mostradas no painel	O servidor ficou pronto antes do previsto. Os testes com dados reais continuam na Etapa 5.

10. Registro de evidências e anexos

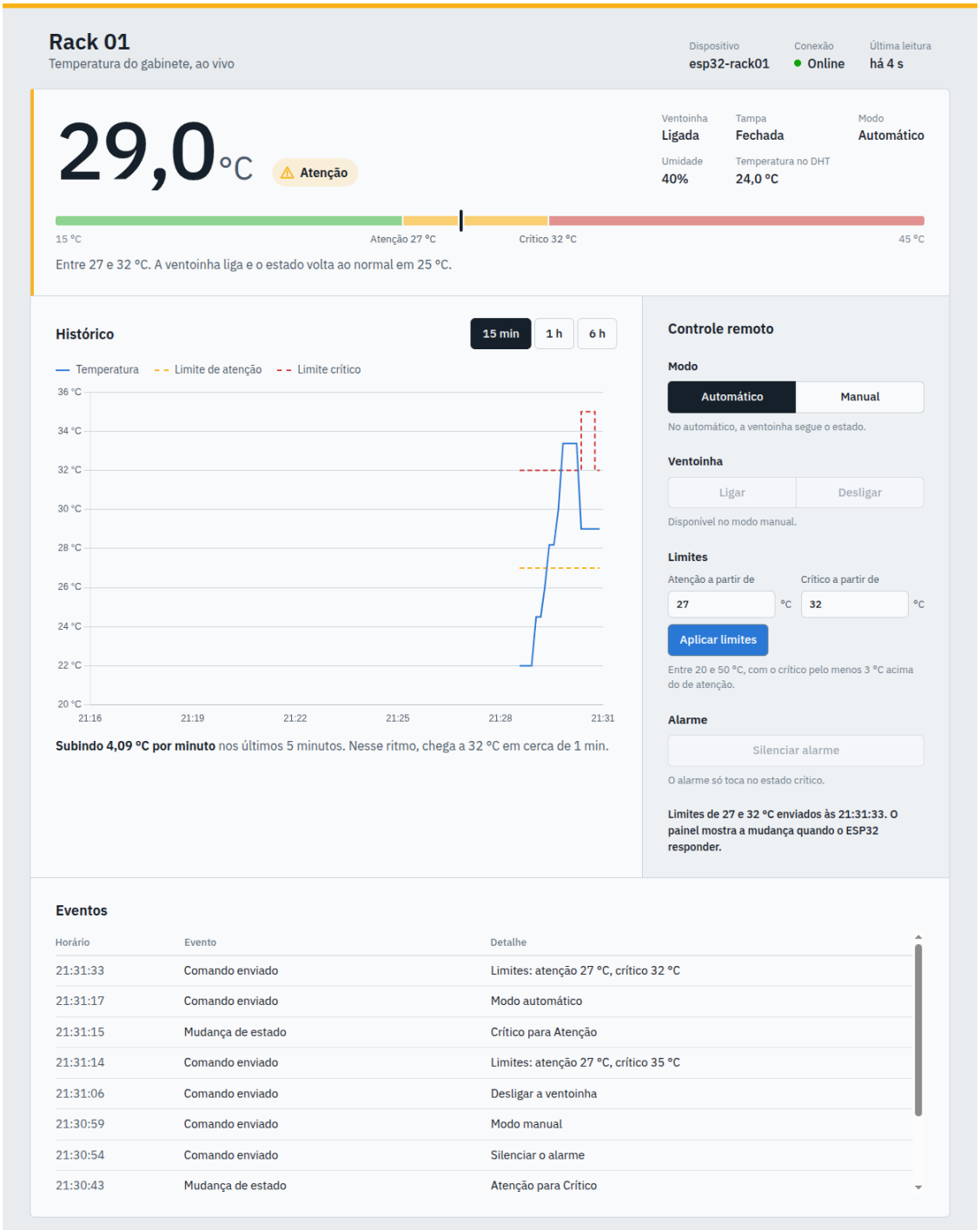
Requisito ou funcionalidade	Tipo de evidência	Arquivo ou link	Como a evidência comprova
Conexão e telemetria (simulação)	Print e link do Wokwi	Anexo A e https://wokwi.com/projects/480262655510684673	O monitor serial mostra `mqtt=on`, com o ESP32 do simulador conectado ao broker.
Painel (simulação)	Print do painel	Anexo B	Mostra a temperatura, o estado, o histórico com os limites, a tendência e os controles.
Comando remoto e validação (simulação)	Tabela de testes	Seção 8	Registra cada comando aceito ou ignorado pelo ESP32 no Wokwi.
Histórico e persistência (simulação)	Gráfico do painel e banco	Anexo B e seção 8	O gráfico vem das leituras gravadas no SQLite: 62 no teste de 09/11 e 37 na repetição de 12/11 (Anexo B).
Formato das mensagens	Tabelas e exemplo JSON	Seção 3	Define tópicos, campos e tipos.
Código atualizado	Repositório	https://github.com/Vinicius-b-oliveira/server-temperature-monitor	Pastas `firmware/monitor`, `server` e `web`.
Conexão do ESP32 à rede	Captura do monitor serial	Anexo C	`Todas as linhas do serial terminam em mqtt=on, com o ESP32 conectado ao broker pela rede da sala.`
Dispositivo e painel ao mesmo tempo	Foto	Anexo E	A bancada e a tela do notebook aparecem no mesmo quadro, e o painel online mostra o estado do ESP32.
Comando remoto no hardware	Print do painel	Anexo D	A tabela de eventos registra os comandos de modo, ventoinha, limites e alarme enviados pelo painel.
Segurança do modo manual	Telemetria gravada no banco	Seção 8, teste 4	Em crítico e no modo manual, depois do comando de desligar, a telemetria continuou com fan_on verdadeiro.
Queda e volta do ESP32	Quadros de vídeo	Anexo F	À esquerda, o painel sem dados novos há 19 s, depois de o USB ser desligado. À direita, o ESP32 de novo online, com a última leitura há 2 s.
Histórico da aula	Print do painel	Anexo D	Gráfico de 1 hora com as leituras da aula e a tabela com os últimos eventos.

Anexo A. Teste de ponta a ponta no Wokwi.



Link do projeto: https://wokwi.com/projects/480262655510684673. O print foi feito no fim do teste repetido em 12/11/2026, com o circuito do Wokwi já igual ao da bancada. O DS18B20 está em 29 °C, o estado é atenção e o monitor serial mostra `mqtt=on`, com o ESP32 do simulador conectado ao HiveMQ.

Anexo B. Painel durante o teste no Wokwi.



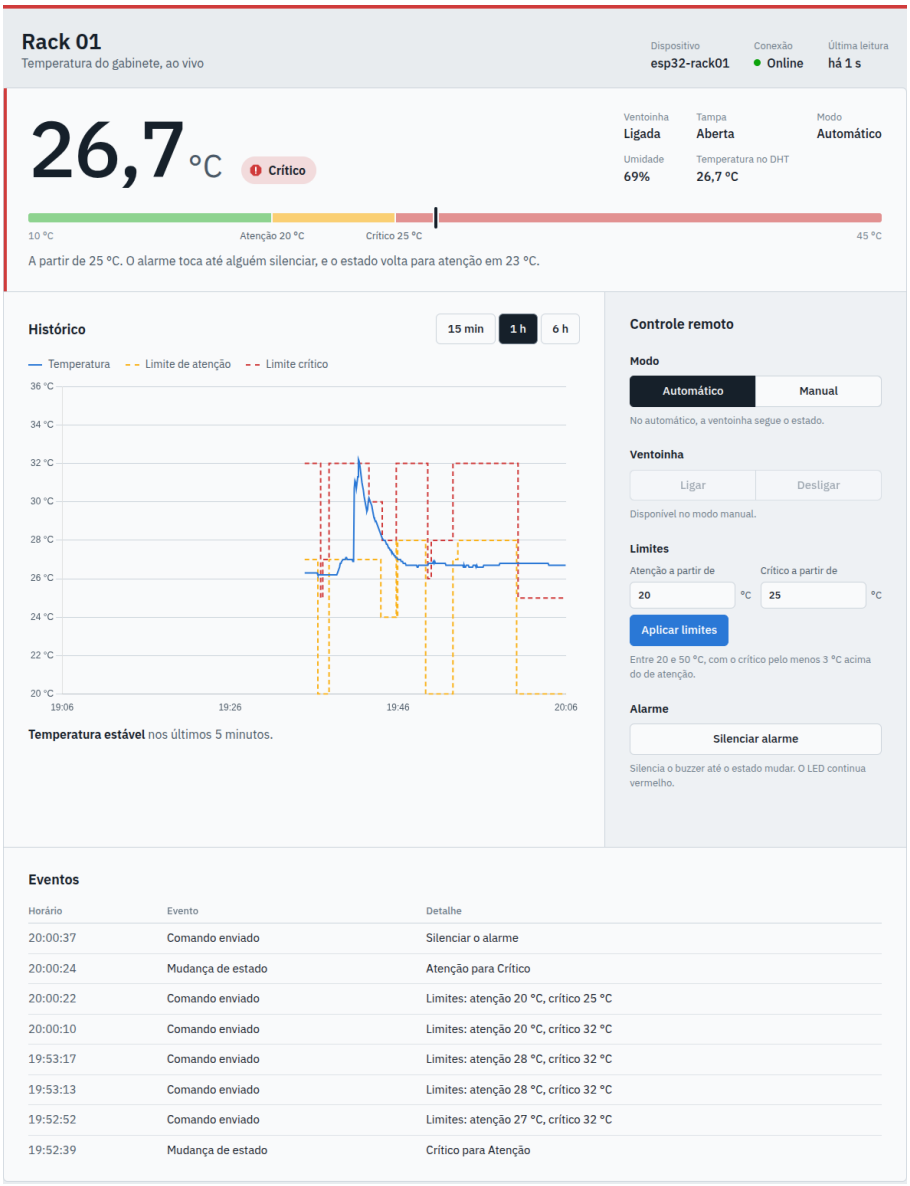
Painel no fim do mesmo teste: 29,0 °C em atenção, ventoinha ligada e dispositivo online. O gráfico mostra a subida de 22 °C até 33,4 °C e a queda para 29 °C, com as linhas dos limites: o crítico foi mudado para 35 °C e voltou para 32 °C. Embaixo, a tabela de eventos registra as mudanças de estado e os comandos enviados pelo painel.

Anexo C. Monitor serial na bancada.

```
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=432 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=438 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=438 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=443 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=448 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=454 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=462 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=464 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=466 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=464 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
temp_c=26.70 dht_temp_c=26.7 humidity=69 ldr=466 lid=open state=critical mode=auto fan=on buzzer=off mqtt=on
```

Recorte do monitor serial do Arduino IDE na bancada, às 20:05, com o firmware monitor. As linhas terminam em mqtt=on: o ESP32 estava conectado ao HiveMQ pela rede da sala. Elas também mostram o DHT11 em 26,7 °C, a leitura do LDR, o estado crítico (com os limites de 20 e 25 °C enviados pelo painel), o modo automático e a ventoinha ligada.

Anexo D. Painel com os dados da aula.



Print do painel no fim da aula, às 20:06, com o gráfico de 1 hora. A linha azul é a temperatura do DHT11, que passou de 32 °C quando o sensor foi aquecido na mão. As linhas tracejadas são os limites, mudados várias vezes pelo painel. A tabela mostra os últimos eventos, com comandos e mudanças de estado.

Anexo E. Bancada e painel ao mesmo tempo.

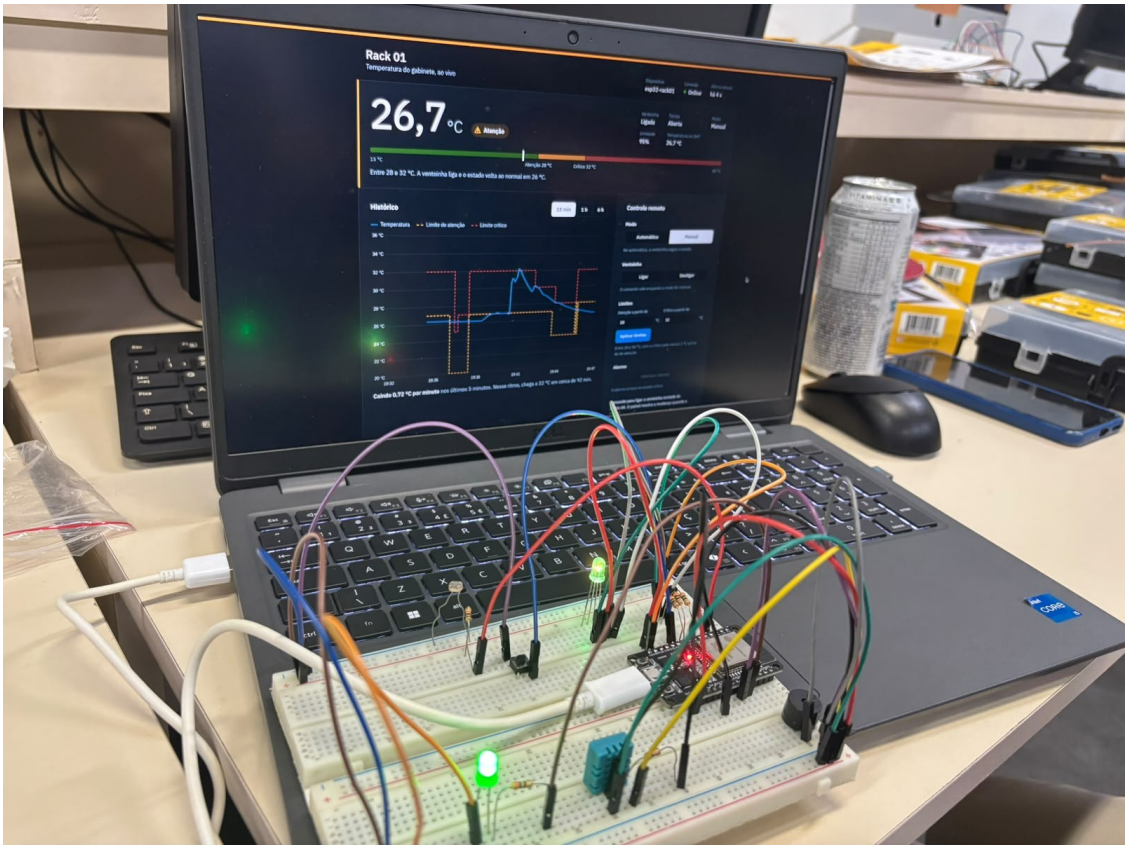
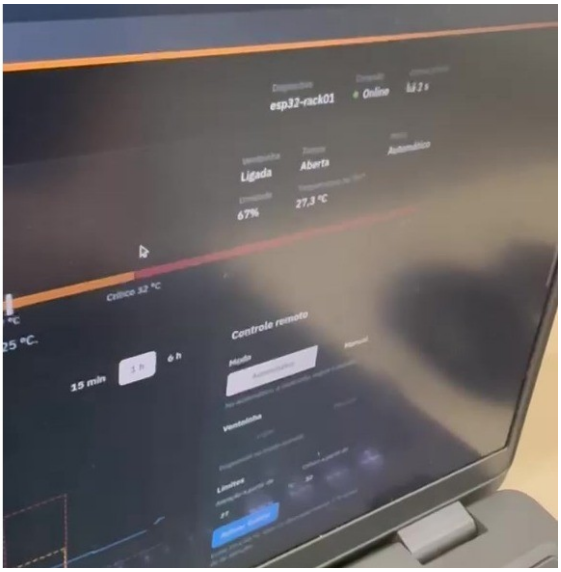


Foto da aula com a bancada na frente e o painel no notebook atrás. O painel mostra 26,7 °C em atenção, e o LED que faz o papel da ventoinha está aceso na protoboard.

Anexo F. Queda e volta do ESP32 na bancada.





Dois quadros do vídeo feito na aula. À esquerda, depois de o USB do ESP32 ser desligado, o painel avisa que não há dados novos há 19 s e mostra a última leitura recebida, 27,2 °C. À direita, com o USB de volta, o ESP32 aparece online de novo, com a última leitura há 2 s e o limite crítico de volta ao padrão de 32 °C.